Modèles Algorithmiques Fondamentaux sur Structures à Accès Direct

Thème principal : Les 3 grands patrons de conception algorithmique de parcours et de recherche sur structures de données à accès direct

Synthèse exécutive : Ce cours établit les schémas réutilisables fondamentaux de la sous-discipline R1.01 (Initiation à l'algorithmique) indispensables pour réussir les devoirs et les SAÉ en BUT 1 Informatique. Avant toute conception d'algorithme, l'étudiant doit analyser le contrat d'interface et le mécanisme d'accès de la collection manipulée (Direct, Séquentiel ou Dynamique). Le cours formalise et décortique trois modèles algorithmiques canoniques sur structures à accès direct (tableaux, chaînes de caractères) : (1) Le parcours complet avec traitement systématique, (2) Le parcours complet avec traitement conditionné, et (3) La recherche de première occurrence (avec ses deux variantes). Une attention rigoureuse est portée à la gestion exacte des bornes d'indices (de 0 à N-1), à l'adoption du principe de pessimisme algorithmique, à l'exigence absolue d'une double condition d'arrêt pour toute recherche, et au choix stratégique de variante selon que l'objectif vise l'obtention d'une position indicée exacte ou la simplicité de codage.

1. Définitions & Typologie des Mécanismes d'Accès aux Données

L'analyse préalable du mécanisme d'accès est l'étape zéro de toute conception algorithmique. Si une structure propose un accès direct (comme un tableau statique de N éléments ou une chaîne de caractères), l'adresse mémoire de n'importe quel élément situé à la position 'i' est calculable immédiatement. En revanche, un flux séquentiel (fichier texte, fichier CSV lu ligne par ligne) n'expose que trois primitives : (1) se positionner en début de flux, (2) lire l'élément suivant, et (3) tester la fin de flux. Il est physiquement et logiquement impossible de sauter directement à la 50ème ligne sans avoir lu les 49 premières. Appliquer une notation par indice de type T[i] sur un fichier séquentiel constitue une erreur conceptuelle récurrente qui invalide entièrement l'algorithme.

- • Structure de données : Collection organisée regroupant des éléments de même type (ex: tableaux, chaînes de caractères).
- • Accès Direct : Capacité matérielle ou logique d'atteindre instantanément n'importe quel élément via un indice i ou une clé (ex: T[i]) en temps constant O(1), sans lire les éléments précédents.
- • Accès Séquentiel : Contrainte imposant de démarrer obligatoirement au début de la structure et de progresser pas à pas via 3 opérations primitives : se positionner au début, passer au suivant, tester la fin (ex: fichiers texte bruts, flux CSV, listes chaînées simples).
- • Accès Dynamique : Structure hybride proposant simultanément des primitives d'accès séquentiel et des mécanismes d'indexation directe.
- • Incompatibilité structurelle : Règle stricte interdisant l'utilisation d'une notation par indice (T[i]) sur une collection purement séquentielle sous peine d'erreur logique majeure.

Cas concret enseignant : *Le professeur insiste sur un piège classique de partiel : tenter d'écrire une instruction d'accès direct T[i] pour lire la i-ème ligne d'un fichier CSV brut ou d'un fichier texte ouvert. Le système d'exploitation ne fournissant pour ces fichiers qu'un curseur séquentiel avance-ligne, l'algorithme est sémantiquement faux et conduit directement à l'attribution de la note zéro sur la question.*

2. Premier Modèle : Le Parcours Complet avec Traitement Systématique

Ce modèle algorithmique s'applique dès lors qu'aucune exclusion ni filtrage n'est permis au sein du domaine d'exploration. Il répond à des besoins comme : initialiser toutes les cases d'un tableau à zéro, afficher le contenu intégral d'une collection, ou convertir chaque caractère d'une chaîne en majuscule. Structure canonique en pseudo-code : Pour i allant de BorneDébut à BorneFin Faire Traiter(Collection[i]) Fin Pour Pour un tableau de N éléments en notation standard (indexation à zéro), la première case se situe à l'indice 0 et la dernière à l'indice N -

1. Le traitement s'exécute de façon indifférenciée et automatique à chaque tour de boucle.

• Objectif : Appliquer un traitement uniforme et inconditionnel à 100% des éléments situés entre une borne de début et une borne de fin.

• Structure itérative : Utilisation obligatoire d'une boucle bornée 'Pour' car le nombre d'itérations est calculable avant l'entrée dans la boucle (Nombre = Fin - Début + 1).

• Gestion des bornes : Sur une collection de N éléments indicée à partir de 0, la variable de boucle 'i' varie strictement de 0 à N - 1.

• Piège critique (Out of bounds) : Faire varier la boucle jusqu'à N provoque une tentative d'accès à la case mémoire T[N] inexistante, entraînant une erreur de dépassement de capacité.

Cas concret enseignant : *Exemple concret donné en amphi : 'Afficher un tableau de N cases'. L'enseignant rappelle que l'erreur la plus fréquente des étudiants débutants est d'écrire 'Pour i allant de 0 à N', ce qui provoque un crash immédiat (out of bounds) à l'exécution lors de la tentative de lecture de la case T[N].*

3. Second Modèle : Le Parcours Complet avec Traitement Conditionné

Le modèle du parcours complet conditionné conserve l'exploration exhaustive du modèle 1, mais encapsule l'action de traitement dans une condition de filtrage. Il est utilisé pour compter des occurrences particulières (ex: compter le nombre de voyelles dans un texte), afficher uniquement un sous-ensemble de données (ex: imprimer les nombres pairs d'un tableau), ou effectuer des calculs cumulatifs ciblés. Structure canonique en pseudo-code : Pour i allant de BorneDébut à BorneFin Faire Si Propriété(Collection[i]) Alors Traiter(Collection[i]) Fin Si Fin Pour Le prédicat logique peut être simple (ex: T[i] > 0), complexe (ex: T[i] > 0 ET T[i] mod 2 = 0), ou comparer l'élément courant avec son voisin immédiat (ex: T[i] > T[i-1]). Dans ce dernier cas, la boucle doit démarrer à i = 1 au lieu de 0 pour empêcher l'évaluation interdite de T[-1].

• Parcours exhaustif : Tous les éléments de la zone délimitée sont inspectés de la borne initiale à la borne finale.

• Traitement sélectif : L'action principale n'est déclenchée que si l'élément inspecté valide un prédicat logique (test booléen).

• Architecture algorithmique : Imbrication d'une structure alternative 'Si ... Alors' au sein d'une boucle bornée 'Pour'.

• Ajustement des bornes : Si la condition compare l'élément courant à ses voisins (ex: i-1 ou i+1), les bornes de la boucle doivent être décalées pour éviter tout déréférencement hors limites (ex: démarrer à 1 si l'on teste T[i-1]).

Cas concret enseignant : *L'analogie de la chaîne de montage en usine donnée par le professeur : Sur une chaîne de fabrication, chaque pièce défile obligatoirement devant l'inspecteur qualité (parcours complet d'inspection de toutes les pièces), mais l'inspecteur n'appose le coup de tampon de rebut (action de traitement) que si et seulement si la pièce présente un défaut spécifique (traitement conditionné).*

4. Troisième Modèle : La Recherche de Première Occurrence & Ses Variantes

Contrairement aux parcours complets, la recherche de première occurrence s'appuie sur une itération non bornée ('Tant que') car le nombre de répétitions est inconnu à l'avance. L'esprit de conception repose sur le principe du pessimisme algorithmique : on pose au départ que l'élément est absent (`trouve <- Faux`). Règle d'or : Rédiger un algorithme de recherche avec une seule condition d'arrêt est une faute grave sanctionnée par la note zéro. Comparatif stratégique des deux variantes d'implémentation : - Variante 1 (Sortie anticipée / Rupture) : Dès que la cible est vue, `trouve` passe à Vrai et la boucle s'interrompt immédiatement. L'indice `i` pointe pile sur la case trouvée. C'est la variante idéale si l'on doit retourner l'indice/position exacte de l'élément. - Variante 2 (Boucle Tant que globale) : La boucle s'écrit `Tant que (i < N et NON trouve) Faire`. Si l'élément est trouvé, `trouve` devient Vrai. Cependant, la boucle effectue l'incrémentation `i <- i + 1` avant de réévaluer la condition de tête. À la sortie, l'indice `i` dépasse la cible de 1 et nécessite un réajustement (`i - 1`). Cette variante est plus simple à coder si seule la validation d'existence booléenne nous intéresse.

• Objectif : Localiser le tout premier élément d'une collection vérifiant une propriété et interrompre l'exploration immédiatement dès qu'il est trouvé.

• Pessimisme algorithmique : Postulat de départ supposant que la cible est absente de la collection (initialisation d'un booléen trouve <- Faux).

• Double condition d'arrêt obligatoire : Un algorithme de recherche valide DOIT s'arrêter (1) si l'élément est trouvé (succès) OU (2) si la structure a été entièrement explorée sans succès (échec).

• Variante 1 (Rupture immédiate) : Sortie anticipée dès le succès ; conserve la variable d'indice 'i' exactement positionnée sur l'élément trouvé.

• Variante 2 (Tant que global) : Condition unique en tête de boucle ('Tant que i < N et NON trouve') ; très propre syntaxiquement mais effectue une incrémentation ultime de 'i' avant de sortir, exigeant un réajustement (i - 1) si l'on veut récupérer la position.

Cas concret enseignant : *La devise du professeur : 'En informatique, soyez pessimistes ! Partez toujours du principe que ce que vous cherchez n'existe pas dans la structure, et ne basculez votre indicateur à Vrai qu'au moment précis où vous le touchez.' Il martèle qu'un algorithme de recherche présenté en examen avec une seule condition d'arrêt (ex: oublier de tester la fin de tableau) recevra systématiquement la note zéro.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le modèle du parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, parfaitement ! Une structure à accès dynamique offre à la fois des primitives d'accès séquentiel et d'accès direct. On peut donc appliquer ce modèle sur une structure dynamique en exploitant spécifiquement ses mécanismes d'accès direct.

Question étudiant : Pourquoi s'embêter à nous enseigner deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : Parce qu'elles répondent à deux besoins distincts en ingénierie algorithmique : la Variante 1 s'arrête immédiatement et laisse l'indice 'i' positionné exactement sur la case de l'élément trouvé (parfait si l'on doit retourner la position exacte). La Variante 2 offre une structure 'Tant que' canonique avec test unique en tête, plus propre à coder si seule l'existence booléenne (Vrai/Faux) nous intéresse.

Question étudiant : Comment fait-on si la condition dans un parcours complet conditionné dépend de l'élément précédent (par exemple T[i] > T[i-1]) ?
Réponse du professeur : Il s'agit toujours du Modèle 2 (parcours complet conditionné), mais l'adaptation exige un ajustement rigoureux des bornes : au lieu de faire démarrer l'indice 'i' à 0, on le fait démarrer à 1. Cela empêche le programme de tenter d'évaluer T[-1], ce qui provoquerait une faute mémoire fatale d'accès hors limites (out of bounds).

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur donnée (par exemple trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Attention au piège ! Dès que l'on veut trouver TOUS les éléments ou tous les indices d'une collection, il ne s'agit plus d'une recherche de première occurrence, mais d'un parcours complet avec traitement conditionné (Modèle 2). On parcourt le tableau de 0 à N-1 et, à chaque fois que la condition T[i] == 5 est vraie, on effectue le traitement (stocker ou afficher l'indice i).

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' elements. Expliciter le modèle algorithmique retenu et la gestion des bornes.

Résolution pas à pas :
Étape 1 : Analyse du problème et sélection du modèle. Il faut inspecter l'intégralité des cases d'un tableau (structure à accès direct) de la première à la dernière. L'action d'affichage ne doit s'exécuter que si la valeur respecte le critère 'strictement supérieur à 0'. Le modèle algorithmique approprié est donc le 'Parcours complet avec traitement conditionné' (Modèle 2). Étape 2 : Détermination des bornes d'indices. En convention informatique standard à indexation zéro, pour un tableau de N éléments, la première case est à l'indice 0 et la dernière à l'indice N - 1. Borne initiale = 0, Borne finale = N - 1. Étape 3 : Structure de contrôle itérative. Le nombre d'itérations est connu

avant d'entrer dans la boucle -> Utilisation d'une boucle bornée 'Pour i allant de 0 à N - 1'. Étape 4 : Écriture du filtre conditionnel et du traitement. À chaque itération i, évaluer 'Si T[i] > 0 Alors' puis déclencher l'action 'Afficher(T[i])'.

Résultat : Algorithme AffichagePositifs Données T : Tableau d'entiers N : Entier (taille du tableau) Variables i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin Commentaire didactique : Cet algorithme garantit un parcours complet exhaustif sans risque de dépassement de capacité (l'itération s'arrête strictement à N-1) et filtre sélectivement les valeurs strictement positives.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique doit-on impérativement employer pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?

R : Le modèle de Recherche de première occurrence (Modèle 3), car il s'agit de vérifier une présence et de s'interrompre dès la première découverte.

Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il obligatoirement N - 1 et non N ?

R : Parce qu'en informatique standard, l'indexation commence à 0. Un tableau de N éléments possède donc des cases indicées de 0 à N - 1. Accéder à l'indice N déclenche une erreur fatale de dépassement de capacité mémoire (out of bounds).

Q : Combien de conditions d'arrêt minimales doit comporter la structure itérative d'une recherche de première occurrence et quelles sont-elles ?

R : Tout algorithme de recherche doit comporter au minimum DEUX conditions d'arrêt : (1) Le succès de la recherche (l'élément recherché est trouvé), et (2) L'échec par épuisement de la collection (la fin de la structure est atteinte).

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765